

DLDCALab 1: Verilog

Week 1

July 31, 2025

Introduction

In this lab we are going to start with a simple set of problems through which we will understand the basics of verilog syntax and semantics. The lab consists of two tasks each with the goal of teaching combinational circuits.

Necessary tools

We shall be using `icarus-verilog` (or) `iverilog` in order to compile verilog HDL (Hardware Description Language). To visualize waveforms we shall use `gtkwave`.

Before starting the lab, ensure that both tools are installed in your system. This can be verified by running the following:

```
$ iverilog
iverilog: no source files.
Usage: iverilog [-EiRSuvV] [-B base] [-c cmdfile|-f cmdfile]
           [-g1995|-g2001|-g2005|-g2005-sv|-g2009|-g2012] [-g<feature>]
           [-D macro[=defn]] [-I includedir] [-L moduledir]
           [-M [mode=]depfile] [-m module]
           [-N file] [-o filename] [-p flag=value]
           [-s topmodule] [-t target] [-T min|typ|max]
           [-W class] [-y dir] [-Y suf] [-l file] source_file(s)

See the man page for details.
$ gtkwave --version
GTKWave Analyzer v3.3.116 (w)1999-2023 BSI
```

```
GTKWAVE | Use the -h, --help command line flags to display help.  
WM Destroy
```

Structure of submission/templates

Submission format:

```
your-roll-no/  
|- task1/  
    |- module.v (TODO)  
    |- tb_module.v
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of your-roll-no.tar.gz. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

Tasks

Task 0: Intro to verilog

First, watch the module-tb-gtkwave.mp4 video to understand how programming is done in verilog. The video also explains how to use gtkwave to visualize waveforms to check correctness of implemented verilog code. Then, go through the data_types.pdf which is a set of slides explaining the difference between wires and registers and how to use them. You should also try to compile, run and visualize the output of the and-gate program mentioned in the video (and_gate.v and tb_and_gate.v).

Task 1: 2 Bit subtractor

The goal of this task is to create a simple two bit subtractor module that takes two input 2-bit vectors (a, b) and outputs a 2-bit vector (sub). The architecture for the 1 bit adder can be seen in figure 1. The truth table for the circuit is as follows:

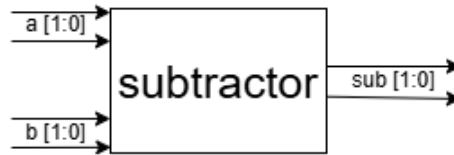


Figure 1: 2-bit subtractor architecture

The truth table for the circuit is as follows:

a[1]	a[0]	b[1]	b[0]	sub[1]	sub[0]
0	0	0	0	0	0
0	1	0	0	0	1
0	1	0	1	0	0
1	0	0	0	1	0
1	0	0	1	0	1
1	0	1	0	0	0
1	1	0	0	1	1
1	1	0	1	1	0
1	1	1	0	0	1
1	1	1	1	0	0

Note that not all combinations of a and b have been shown in the truth table, for those values that are not shown, you may assume any output is okay.

Derive the formulae for the sub[1] and sub[0] wires from the truth table, using karnaugh maps.

You have been provided a template module file (module.v), you are expected to fill in the logic to complete the subtractor. In order to test your implementation we have also provided you a testbench (tb_module.v).

To run your code, first compile the module using:

```
$ iverilog -o testbench tb_module.v module.v
```

After this run the testbench and see the output in gtkwave using

```
$ vvp testbench
$ gtkwave wave.vcd
```

Make sure to go through the testbench code to understand how to output your testbench output to .vcd files in order to visualize them with gtkwave.